Four Mental Models

Week 1 was a tour of `id4me.user.service`. The tour isn't the lesson. What transfers to **any** backend is four questions you learn to ask: **where are the boundaries, which way do dependencies point, how long do things live, and how does a service say no?** Everything here is read from the code on `dev`, not from the curriculum.

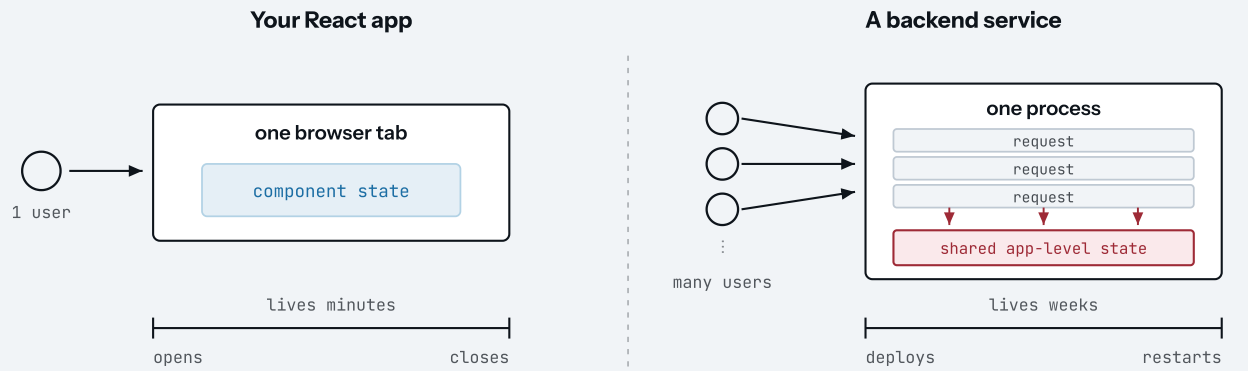
> WEEK 1 GOAL — Name every box in the ecosystem and say who owns it · run both APIs locally with a green health check. The first half landed; the second is still open — see the last sheet.

START HERE

A server is not a browser

EVERYTHING ELSE FOLLOWS FROM THIS

One tab and one user, versus one process and hundreds of users at once. Every difference in this document comes from that.



The red arrows are the whole difference. In the browser, a variable held at module level is private to one person. In a service, three requests running at the same moment all reach the same variable — so state held at app level is shared by strangers, and it survives until the process restarts.

- **Shared state becomes a hazard,** not a convenience. This is the reason lifetimes exist later in this document.
- **Nothing is disposable.** The process outlives the request, so anything kept in memory persists — and vanishes on restart. Sessions in this service work exactly that way, which is why a restart logs everybody out.
- `async` / `await` **looks identical but isn't.** Same keywords; work runs on thread-pool threads rather than one event loop. The parallelism is real, so the concurrency bugs are real.
- **There's a compile step.** No Vite-style hot reload. Hot reload exists within limits; some edits demand a restart.

→ *So the first question is: what's actually inside that process that we control?*

FIRST, THE WORDS

How objects work here

NEEDED FOR EVERYTHING AFTER THIS

Five words that the rest of this document leans on. In React you rarely think about object creation; here it's the whole mechanism.

`class`

A blueprint. `class User { }` describes the shape of a thing. Same as TypeScript.

`instance`

One actual object built from the blueprint. `new User()` makes one, with its own values inside it. **Two instances are two separate objects that share nothing.** *Every lifetime question later is only ever: how many of these exist, and who shares them?*

`constructor`

The method that runs when an instance is created, and where a class receives the things it needs to do its job. *This is the hook dependency injection plugs into.*

`property`

`public string Email { get; set; }` — looks like a plain field, but is really a pair of tiny methods. `WEEK 5 PREVIEW` this is *why* the database layer can notice when a value changes.

`interface`

A promise about what something can do, with no code for how. `IUserRepository` says "there is a way to get a user by email" without saying where the data comes from. **A leading `I` always means interface.** *This is the keystone of the whole architecture — it comes back on sheet 6.*

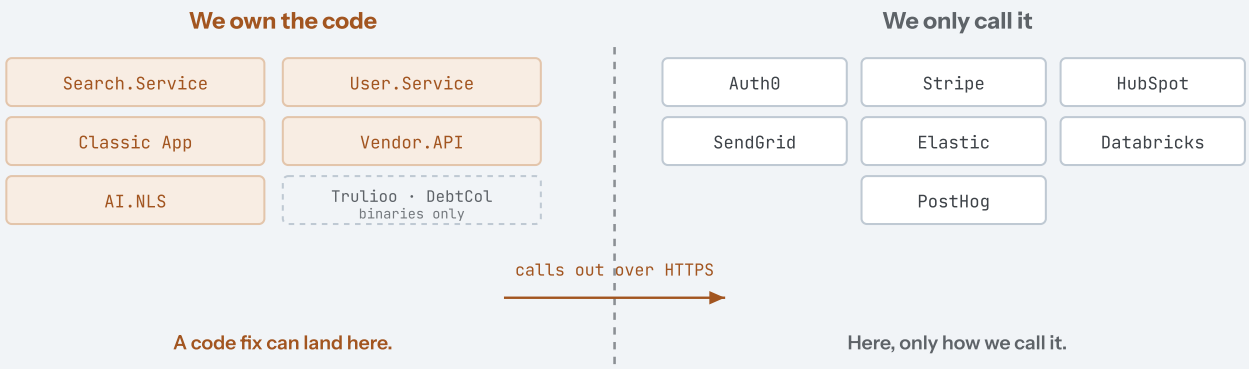
One real divergence from TypeScript. In TS, any object with a matching shape satisfies an interface. In C# a class must explicitly declare that it implements one — `class UserRepository : IUserRepository`. Matching by accident isn't enough, and that single rule explains most of the extra ceremony you're about to see.

→ *Now the words exist. Next: which pieces of this system are ours to change?*

01 Boundaries — what can a fix touch?

MODEL ONE OF FOUR · THE OUTER BOUNDARY

Mark's opening question, and the one to ask on every ticket: **do we own the code, or do we only call it?**



The dashed line is the thing to remember. Left of it is code on our disk. Right of it is somebody else's running service. That decides the shape of every fix: on the left you change behaviour, on the right you can only change the call, the retry, or the fallback. Two footnotes — the User DB is slowly becoming the source of truth for users, taking over from Auth0; and Trulioo/DebtCol sit awkwardly on our side because they reach our data through an app we only hold decompiled binaries for.

→ *That's the boundary around the whole system. There's a second one inside a single service.*

01 The four projects of `user.service`

THE INNER BOUNDARY

One service splits into four projects, each compiled separately. The names are unhelpful at first, so here is what each one actually holds — with real files and real sizes.

id4me.User.Api — 15 files

The front door. Requests from the outside world arrive here. It checks the caller is allowed in, hands the work off, and turns the answer into an HTTP response. Deliberately thin.

`Controllers/PlansController.cs` · `Middleware/` · `Program.cs`

≈ your route handlers

id4me.User.Application — 122 files

The rules. Decides what should happen and in what order: what a signup does, when an OTP is still valid, which plans a lead gets shown. Never talks to Stripe or the database itself. **This is where the work is.**

`Services/PaymentService.cs` · `Services/OtpService.cs` · `Validators/` · `DTOs/`

≈ your business logic, with the fetch calls taken out

id4me.User.Domain — 32 files

The nouns. The things the product is about — a user, a plan, a subscription, a lead who hasn't paid yet. Just their shape, no behaviour. Also keeps a wishlist of jobs it needs done, without saying who does them.

`Entities/User.cs` · `Entities/Plan.cs` · `Entities/StagedUser.cs` · `Interfaces/IUserRepository.cs`

≈ your types.ts

id4me.User.Infrastructure — 63 files

The plumbing. The only project that talks to anything outside the process. Every file here has a wire attached.

`Clients/StripeApiClient.cs` · `Clients/Auth0Client.cs` · `Repositories/UserRepository.cs` · `Migrations/`

≈ your api-client layer

> **Two of these names actively mislead — worth saying out loud.**
>
> **"Application" sounds like it means the whole app.** It doesn't; it's only the rules. `Program.cs`, the file that actually starts the application, lives in **Api**.
>
> **"Domain" sounds technical.** It isn't. It means the business's own vocabulary — the words a salesperson would use. User, plan, subscription, lead.

What the classes are called

HOW TO NAVIGATE THE FILE TREE

The words on every diagram from here — Controller, Service, Repository — are job titles. Here's the job each one does.

Controller

The class that receives HTTP requests. One class per area of the API, one method per endpoint — `PlansController.GetPlans()` runs when the UI hits `GET /plans`. *≈ your route handler.*

Service

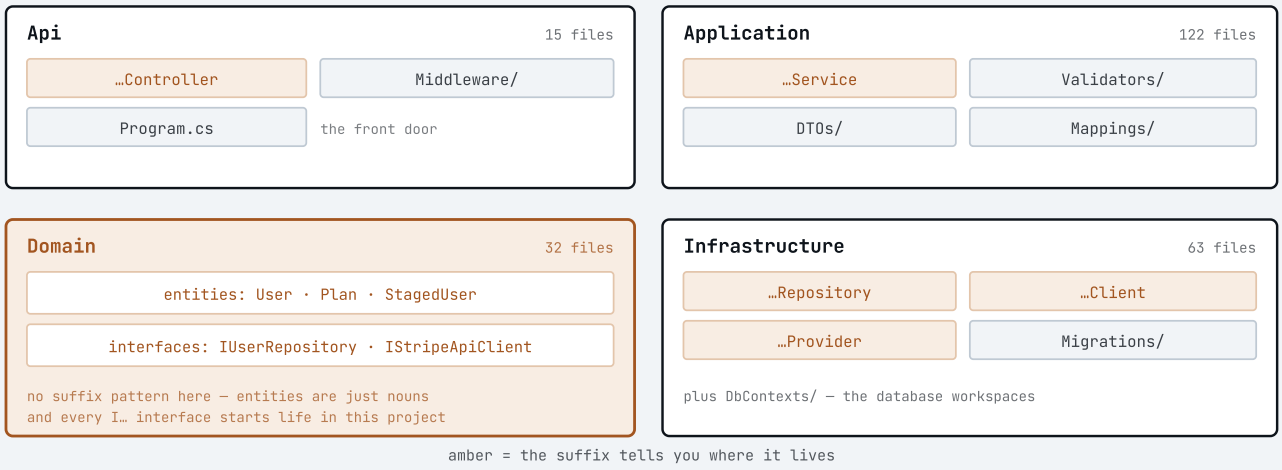
A class holding the rules for one area of work. Controllers call them; they call repositories and clients. The name just means *a class that does work, named after the work.*

Repository

A class whose only job is getting things in and out of the database. Roughly one per entity, and nothing else in the app touches the database directly. *≈ the one module in your app allowed to call fetch.*

Client
Provider

A class whose only job is talking to one external system — `StripeApiClient`, `Auth0Client`. Same idea as a repository, but over HTTP. **Provider** is this repo's word for the ones that send something out: `SendGridEmailProvider`, `MessageMediaSmsProvider`.



amber = the suffix tells you where it lives

Read a filename, know the project. A file ending `Controller` is in Api. `Service` is in Application. `Repository`, `Client` and `Provider` are all in Infrastructure. Domain is the exception with no suffix pattern — its entities are plain nouns, and it's where every `I…` interface is declared.

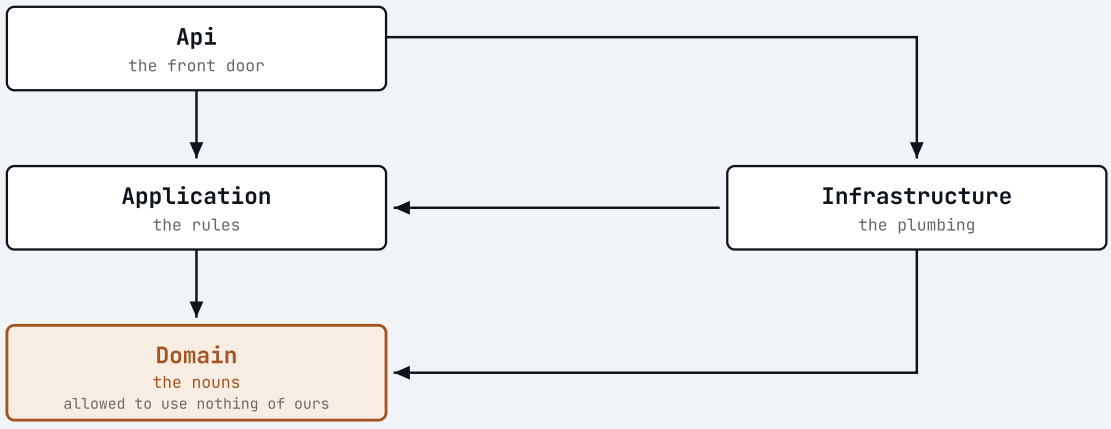
A heuristic, not a law. This repo has `SearchService.cs` sitting in `Infrastructure/Clients` despite the `Service` suffix, and an `Auth0Service` in Application alongside an `Auth0Client` in Infrastructure. **When the suffix and the folder disagree, the folder is the truth.**

→ *Now you know the pieces and their names. What are they allowed to do to each other?*

02 Direction — which way do dependencies point?

MODEL TWO OF FOUR

Each project declares which others it is allowed to use. Those declarations are the architecture — not the folder names.



every arrow reads “is allowed to use”

Api is allowed to use everything, which is how it wires the app together. **Domain is allowed to use nothing of ours** — and that's the point: because it depends on no one, anything can depend on it without dragging the rest of the service along.

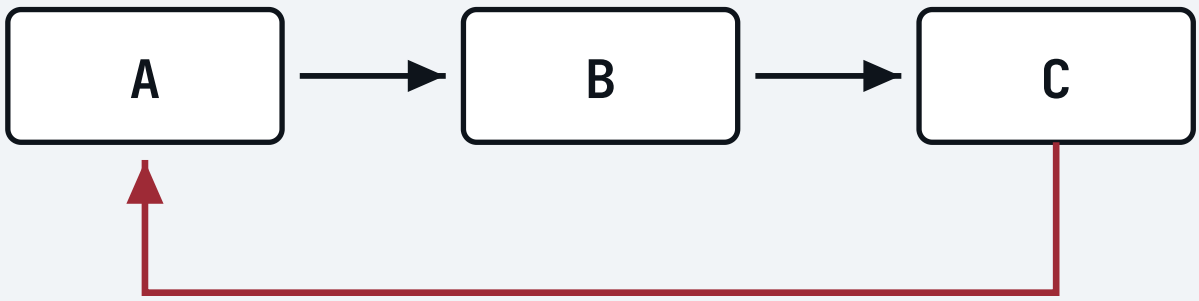
Arrows are declared once per project, in its `.csproj` file, not per file — so no single file can sneak past the boundary. It's the same relationship as a dependency in `package.json`: the manifest grants access, and code inside the project can then use it.

You already sort front-end code into components, hooks and an api-client. Same instinct. The difference: there, an import from the wrong layer still compiles. Here it won't build.

This shape is not universal. `search.service` has no Domain or Application project at all. It's `Api · Services · Data · Model · Contracts · Common`, plus two older projects called `Search2Dal` and `Search2Services` that hold most of its code. Don't go looking for `Domain` over there.

The one shape that can't exist

CIRCULAR DEPENDENCY



nothing can be built first

so the build fails

A circular dependency is when A is allowed to use B, and B is allowed to use A — directly, or round a longer loop like this one.

Why it's impossible rather than discouraged. Each project compiles into its own separate file, and to compile one the compiler needs the finished output of everything it uses. To build A it needs B finished; to build B it needs C finished; to build C it needs A — which hasn't started. There's no order in which the build can happen.

Two different things protect you, and they're worth telling apart. The **compiler** makes a circular dependency impossible — it cannot produce output. Separately, **Rider** is being helpful: when you open the Add Reference dialog, choices that would create one are simply not offered.

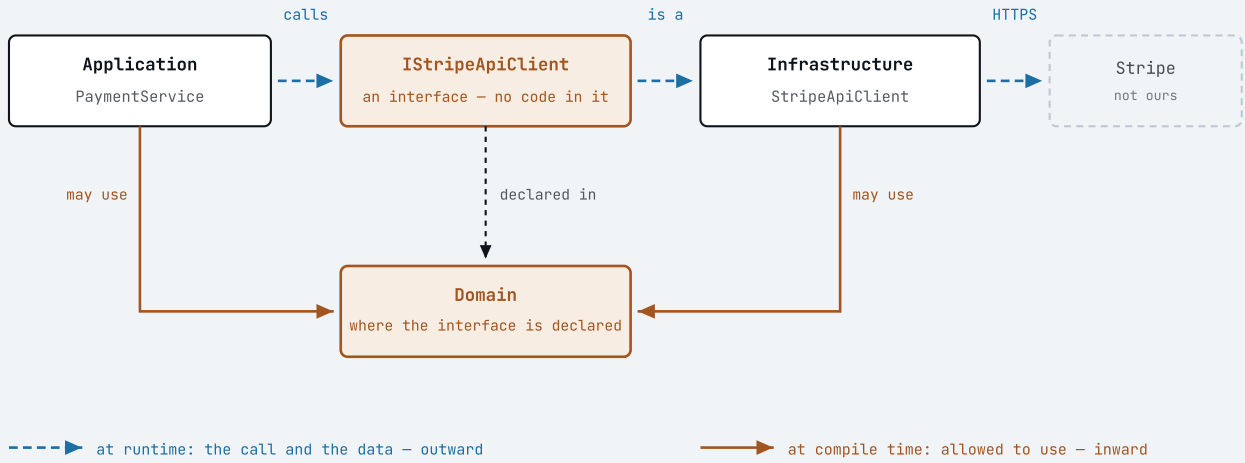
> **You have hit this before — silently.** TypeScript compiles everything into one bundle, so circular imports between files are legal there. You don't get a build error; you get the mystery bug where an import is `undefined` at runtime in whichever file happened to load second. Here the same shape can't get past the build.

→ *The arrows say who may use whom. But Domain is allowed to use nothing — so how does it ever reach Stripe?*

02 The inversion

THE KEYSTONE · SPEND TIME HERE

Two arrows run between the same two boxes, in opposite directions. Data flows outward to Stripe. The permission to use still points inward.



The blue arrows and the amber arrows point opposite ways. That's the entire idea. Read the top row left to right: `PaymentService` calls a method on `IStripeApiClient`, and the object sitting behind that interface *is* a `StripeApiClient` — supplied at runtime by the container. Application never names Infrastructure anywhere.

Which is why the two diagrams don't contradict each other. On the previous sheet, the arrow ran `Infrastructure → Application`. Here the call appears to run the other way. Both are true: the *call* goes out through an interface Application borrowed from Domain, while the *permission* still points inward. (`Infrastructure → Application` exists for a mundane reason — `StripeApiClient.cs` opens with `using id4me.User.Application.DTOs.Models;`, because it returns Application's DTOs.)

Payoff one — you can work in Application without learning Stripe. Open `PaymentService.cs` and the only Stripe-shaped thing in it is the name `IStripeApiClient`. No SDK, no API keys, no webhook semantics. The entire Stripe surface stays out of your head while you change the rules.

Payoff two — tests don't need the internet. `PaymentService` holds an interface, so a test hands it a stand-in that returns whatever the test wants. No test ever calls Stripe. You can only do this because the interface exists.

Worth knowing before you trust the picture. In this codebase Domain is not actually sealed off. `id4me.User.Domain.csproj` references `Stripe.net`, `Auth0.ManagementApi`, EF Core and Npgsql, and `IStripeApiClient` takes a `Stripe.Invoice` and returns a `Stripe.Customer`. The *shape* above is real and worth learning; the isolation it implies is partial.

→ *These arrows say who may use whom. But somebody has to actually build these objects — who, and when?*

03 Who builds these objects?

MODEL THREE OF FOUR · FIRST, THE CONTAINER

Before lifetimes can mean anything, two things have to be on the table: what a request is, and who is doing the building.

a request

One HTTP call, with a beginning and an end. The framework opens a fresh workspace when it arrives and closes it when the response goes out. That window is what "per request" means below.

Constructor injection is props for classes. A React component doesn't create its own props — the parent passes them in. That's exactly why you can test a component by passing fake props. Same mechanism here: a class lists what it needs in its constructor, and something above it supplies them.

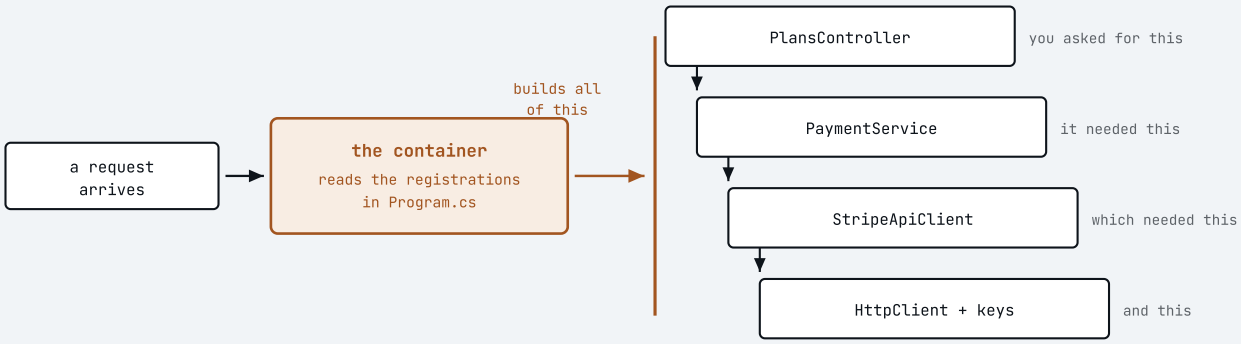
That something is **the container**. It has two halves, and they live in different files:

Once, in `Program.cs` — what to use:

```
AddScoped<IUserRepository, UserRepository>();
AddScoped<IPaymentService, PaymentService>();
```

Everywhere else — what you need:

```
public PlansController(
    IPaymentService paymentService)
// never new PaymentService(...)
```



This is the part with no front-end equivalent. You never assemble that chain. The controller says it needs a service, the service says it needs a client, the client says it needs an HttpClient and some keys — and the container works out the order and builds all four. That's also why swapping one line changes everything below it:

```
AddScoped<IUserRepository, UserRepository>();      // today
AddScoped<IUserRepository, MySqlUserRepository>(); // one line changed
```

Every class that asked for `IUserRepository` now receives the other one, and not one of them can tell.

→ *So the objects get built for you. The remaining question is how long each one sticks around.*

03 Lifetime — how long does this thing live?

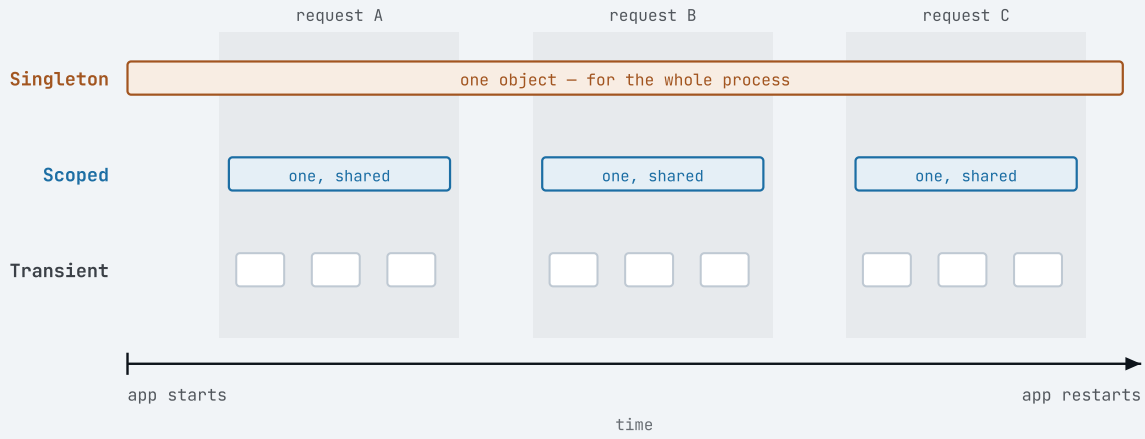
BUILD FRESH, OR HAND OVER ONE YOU HAVE?

A direct consequence of one process serving many users at once.

So the question a lifetime answers is just this: when something asks the container for an `IUserRepository`, should it **build a fresh one**, or **hand over one it already built?** There are three possible answers, and that's all a lifetime is.

REGISTERED AS	WHAT THE CONTAINER DOES WHEN ASKED	USED HERE FOR
Transient	Always builds a fresh one. Ask twice, get two separate objects.	cheap, short-lived objects
Scoped	Builds one per request, then **reuses that same one for the rest of that request**. Next request gets a new one.	anything holding a database workspace or per-user state
Singleton	Builds one the first time it's ever needed, then hands that same object to every request forever — until the app restarts.	stateless helpers, caches, API clients

The part that makes "Scoped" click. "One per request" hides the point. During a single request, several different services each ask the container for `IUserRepository` — and **they all get the same one**. That sharing *is* the value: one request gets one workspace, so everything that happens during it belongs to the same piece of work. It is not "a new one every time"; that's Transient.



Read the bars against the shaded request bands. The singleton bar never breaks. The scoped bars start and end with their request. The transient boxes appear three times *inside* one request. In `Program.cs` today there are 10 scoped registrations and 27 singletons.

→ *So why does it matter which one you pick? Because of what some of these objects are holding.*

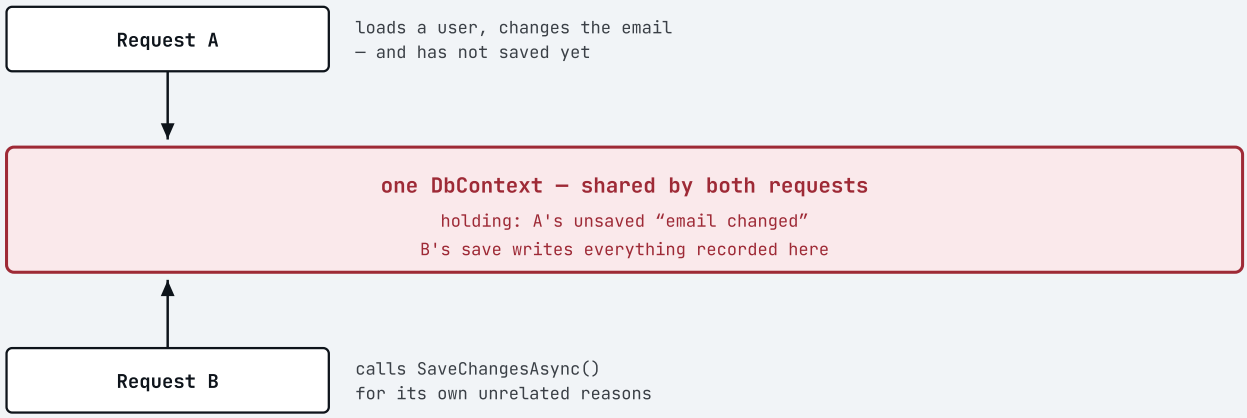
03 Why the choice matters

WHAT GOES WRONG WHEN THE LIFETIME IS WRONG

Why this sheet exists: a lifetime looks like a performance setting — how often do we rebuild this thing? It isn't. It's a correctness decision. Pick wrong and you write the wrong data to the database. Here is the object that makes it so, and the two ways people get there.

WEEK 5 PREVIEW **A `DbContext` is a workspace, not a connection.** Load a user through it and it keeps a copy and watches it. Change a property and it records *"email changed"*. Call `SaveChangesAsync()` and it writes every change it has recorded, all at once.

That one fact — **it holds unsaved work in progress** — is what makes sharing one between requests dangerous.



B's save writes A's half-finished change too — at a moment A never chose, and possibly in a state A would never have committed. Separately, a `DbContext` isn't safe to use from two places at once, so you also get intermittent crashes about a second operation starting before the first finished. That is why `IUserRepository` is registered **Scoped**: one request, one workspace, thrown away at the end.

Two ways to end up there

Both mistakes have the same destination — **one `DbContext` shared by every request**, exactly as drawn above. They just take different routes.

Route A — register it Singleton. The obvious one. One repository object for the whole process, so one workspace for every request that ever arrives.

Route B — register it Scoped, then inject it into a Singleton. The subtle one. The registration is right, but a singleton grabs one instance and never lets go.

Route B is the stale closure, and you've had this bug in React. A long-lived thing captures a snapshot of a short-lived thing and freezes it — a callback still confidently using a value from three renders ago. Same shape, three steps:

- **A singleton is built once**, so its constructor runs once.
- **Whatever the container puts into that constructor is what the singleton keeps** — forever.
- **If one of those things was scoped**, a per-request object is now permanent, shared by every request that ever arrives.

The rule that prevents both. A singleton must never hold a scoped dependency. The other direction is safe: a scoped thing may depend on a singleton. *If someone calls this a "captive dependency", this is what they mean.*

Good news you'd otherwise learn the hard way. In the Development environment, .NET refuses to start and tells you — it validates that no singleton is consuming a scoped service. In other environments that validation is off, so the app starts fine and quietly misbehaves. So you'll usually meet this as a startup error while you're working, not as a production mystery.

→ *Objects now exist and can do work. What happens when the work fails?*

04 Failure — how does a service say no?

MODEL FOUR OF FOUR

First, what "the stack" means. When `Controller` calls `Service` calls `Repository`, each one is sitting there waiting on the next. That's the stack. If the repository **throws**, it stops, then the service stops, then the controller stops — all the way up until something catches it. **Nobody in between gets a say.** That's what "unwinding" means.

A service says no the same way `useQuery` says no — it hands back `{ data, error }` and makes you look. Throwing instead is like letting a fetch blow up into an error boundary: technically it's handled, but nobody in between got to choose what the user sees.

What you already write:

```
const { data, error } = useQuery(...)
if (error) return <Message/>
// now data is safe to use
```

The same move here:

```
var result = await _service.GetPlansAsync(id);
if (result.IsError) return ...;
// now result.Data is safe to use
```

`Result<T>` **is** `{ data, error }`. Same shape, same discipline: the thing that can fail hands back a wrapper, and you check the error before you're allowed to trust the data.

```
// can this fail? nothing in the signature tells you
Task<User>         GetUserAsync(string email);

// failure is part of the return type — the caller must look
Task<Result<User>> GetUserAsync(string email);
```

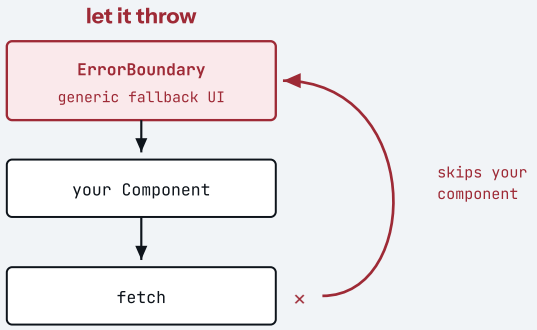
A throw is **invisible in the signature**. The caller has no way to know it's coming, so it can't be made to handle it. `Result<T>` puts failure in the return type, so the compiler makes you check.

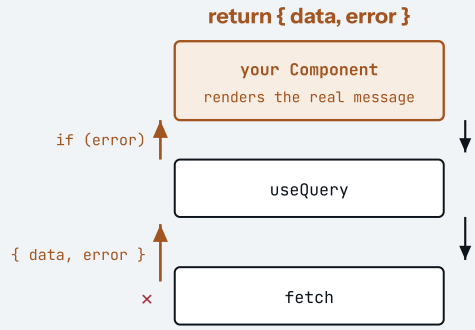
What `Result<T>` actually holds

`IsSuccess / IsError`	which one this is
`Data`	the value, or null
`Error`	an `ErrorResult`, or null

What `ErrorResult` holds

`Code`	`"Session.InvalidHeader"`
`Message`	human-readable text
`Type`	**picks the status code**
`Details`	field → messages, for validation



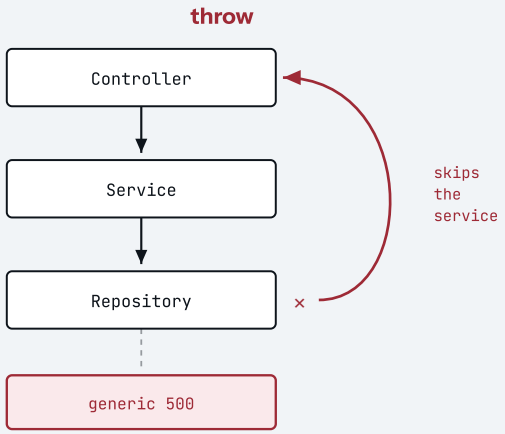


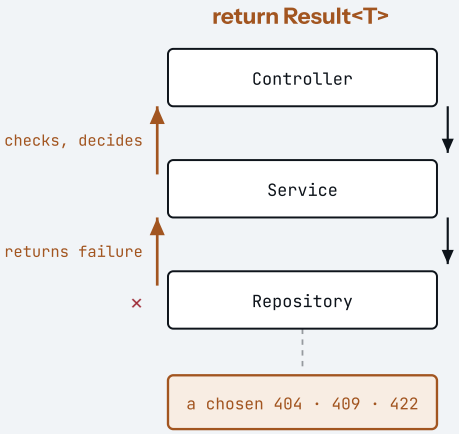
You already make this choice every week. The left half is what happens when a fetch throws and nothing catches it: the user gets the fallback screen, not the message you would have written. Read the backend version straight off it — **error boundary → the global exception handler**, **fallback UI → a generic 500**, `{ data, error }` → `Result<T>`.

> **And `Details` is your form errors.** `ErrorResult.Details` is `field → messages` — the same shape you already use to put "email already taken" under the email input instead of in a toast. Not a new idea, just a named field for it.

04 The same two choices, in C#

THEN: WHO PICKS THE NUMBER?





Left, nobody chose the outcome. Right, somebody chose at every step. In practice you'll see guard clauses stacked at the top of a method, each returning early — no session, no staged user, no country. Error codes are dotted strings kept in one constants file, never written inline.

> **Heads up — there are two different `Result<T>` types in this codebase.** Services return the hand-rolled one, `Application.Common.Base.Result<T>`, with `IsError` / `Data` / `Error`. But `IStripeApiClient` returns `LanguageExt.Core`'s `Result<T>`, which you consume with `.Match(Succ:, Fail:)`. `PaymentService` converts between them — that's what the `plans.Match(...)` call is doing. Two Result APIs one file apart; you're not going mad.

Who decides the status code?

THE SERVICE DOES · READ FROM RESULTMAPPINGEXTENSIONS

The service picks a `ReturnStatus` when it builds the failure. The controller's `ToActionResult` only translates it. This is the lookup table.

SERVICE CHOOSES	CALLER GETS
`Validation`	422
`BadRequest`	400
`Unauthorized`	401
`Forbidden`	403
`NotFound`	404

SERVICE CHOOSES	CALLER GETS
`Conflict`	409
`MethodNotAllowed`	405
`TooManyRequests`	429
`Error / Unexpected`	500
`success, no data`	204

The two that will trip you up. Validation failures here return **422**, not the 400 you'd expect. And a successful result whose `Data` is null returns **204 No Content**, not 200 with an empty body. Both are decided in one `switch` in `ResultMappingExtensions.cs`.

Expected or unexpected? A test you can actually apply

The convention is that expected failures are returned and unexpected ones are thrown. The way to tell them apart:

Would you write a test asserting this outcome?

Yes → return it. It's a normal outcome of a working system. *User not found. Email already taken. Session expired. Card declined.*

No → let it throw. Nobody planned for it and there's nothing sensible to return. *Database unreachable. A bug. A null that shouldn't be possible.*

Throwing for an expected outcome is slow and hides intent — and as the signature comparison showed, it takes the decision away from every caller in between.

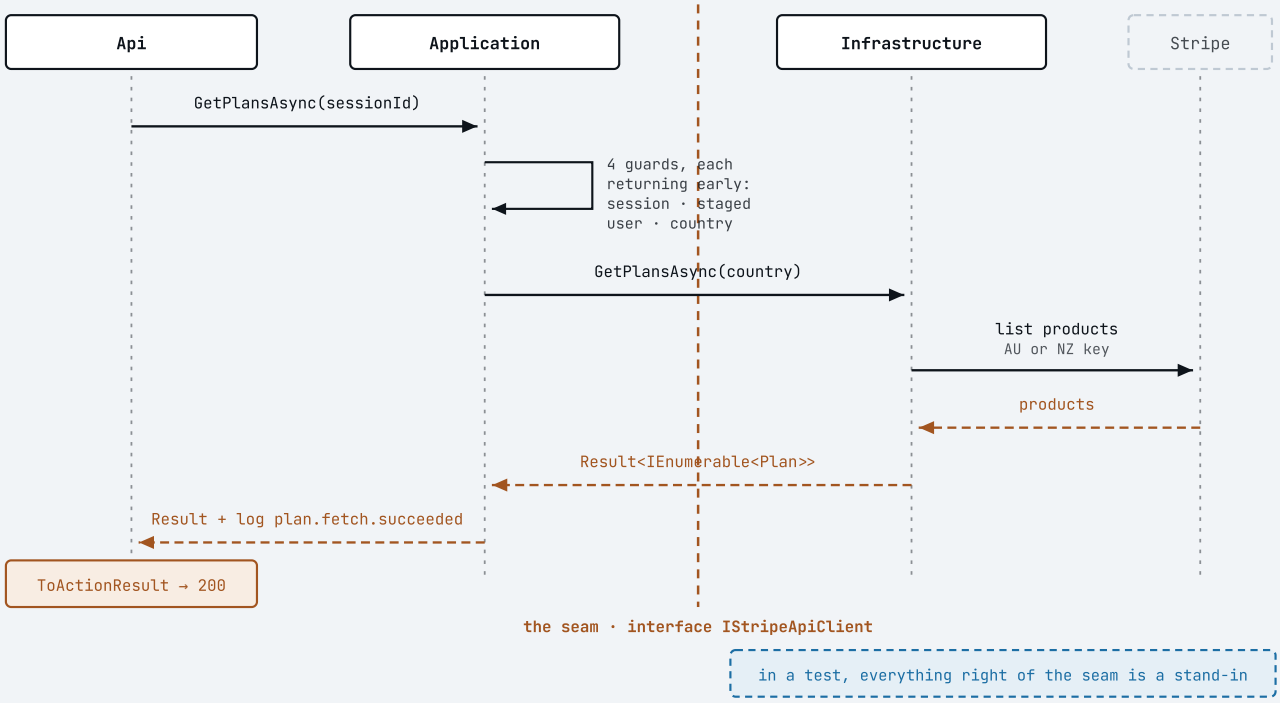
→ *You've now met all four models separately. Here they are moving at once.*

PUTTING IT TOGETHER

One real request: `GET /plans`

ALL FOUR MODELS AT ONCE

Everything so far has been static — boxes, rules, names. This is all of it moving on one real request: the signup UI asking which plans to show. **Being able to trace a request cold is how you'll size a ticket before touching it.**



All four models are visible here. The **boundary** is the dashed amber line — our code on the left, the outside world on the right. The **inversion** is what lets that line exist: the crossing happens through an interface. **Lifetime** decides that the objects doing this work are built fresh for this request and thrown away after. **Failure** is why the return trip carries a `Result` rather than an exception.

The seam is also where the architecture pays for itself: because the crossing is an interface, a unit test replaces everything beyond it, sets up what the stand-in returns, and asserts — with no network. Note too that the Stripe call picks the AU or NZ key by country, so every Stripe change must be considered for both.

And the same move on a ticket you haven't seen

Take the curriculum's own exercise — *"store a user's preferred contact method"*. Before writing anything, the four projects tell you where it lands:

PROJECT	WHAT THIS TICKET NEEDS THERE
Domain	add the property to `User.cs`
Infrastructure	a migration to add the column
Application	a DTO field, a validator rule, the service that sets it
Api	nothing — unless it needs a new endpoint

→ *That's the move you'll repeat on every ticket. The rest is reference.*

REFERENCE

Translation card

ADD A ROW EVERY WEEK

YOU KNOW		HERE IT'S
package.json	→	Foo.csproj
a package.json dependency	→	<ProjectReference>
workspace root	→	Foo.sln
npm / node_modules	→	NuGet / obj + bin
npm install	→	dotnet restore
npm run build	→	dotnet build
npm run build → dist/	→	dotnet publish
npm run dev	→	dotnet run · ▶ in Rider
main.tsx + App.tsx	→	Program.cs

YOU KNOW		HERE IT'S
routes.ts	→	Controllers/
types.ts	→	Domain/Entities/
Promise<T>	→	Task<T>
decorators	→	[Attributes]
zod schema	→	FluentValidation
neverthrow Result	→	Result<T>
jest.mock()	→	new Mock<IThing>()
useContext	→	constructor injection

There is no `scripts` block. `restore`, `build`, `run`, `test` and `publish` are built-in `dotnet` verbs fixed by the SDK — not aliases someone defined in the project file. You can't invent `dotnet dev`.

Three bits of C# syntax you met in passing. `[Attributes]` in square brackets are metadata the framework reads and acts on — `[Route]`, `[HttpGet]`, `[FromHeader]`, `[Authorize]`. **Extension methods** are a `static` method whose first parameter is `this ISomething x`, which makes it appear on every `ISomething` — that's why `AddSwaggerGen()` and `ToActionResult()` read like built-ins. **Anonymous objects** — `new { status = "healthy" }` — are a plain object literal with no declared class, which is what the health endpoint returns.

Items of interest

FOUND IN THE CODE — WORTH A LOOK, NOT A FIX

Key Vault is switched off

The Key Vault block in `Program.cs` is commented out, so configuration comes from environment variables and `appsettings.json` — and that file currently holds ten populated live credentials, which are also in git history.

Validation isn't firing

Both validation middlewares match paths starting `/api/registration/…`, but `RegistrationController` is routed at `registration`, with no `/api` prefix. The three tests that would catch it are commented out.

Two services, two conventions

`search.service` does the same jobs differently — `ServiceResult<T>` instead of `Result<T>`, Newtonsoft instead of System.Text.Json, Elasticsearch instead of Postgres. Follow the local convention, not the one you learned first.

Domain isn't actually isolated

`id4me.User.Domain.csproj` references `Stripe.net`, `Auth0.ManagementApi`, EF Core and Npgsql, and `IStripeApiClient` takes a `Stripe.Invoice` and returns a `Stripe.Customer`. The pattern says the innermost layer knows nothing of the outside world; here it knows quite a lot. A good question for Mark.

Where this leaves you

BEFORE WEEK 2

- **Landed:** the ecosystem and ownership · the four projects and the dependency rule · the inversion · the container and lifetimes · `Result<T>` and status mapping · a unit test with a stand-in.
- **Skimmed:** C# language basics — that's week 2's whole session, so the gap is expected.
- **Not covered:** EF Core, migrations and SQL · JWT validation and what `[Authorize]` actually checks · integrations, retries and idempotency.

> STILL OPEN Neither API has been run locally yet — that was the actual Week 1 check, and every hands-on session from here assumes it. Ask Mark how local configuration is meant to be supplied, given the Key Vault block is off.